

Fájlkezelés C#-ban — Összefoglaló

Ez az anyag bemutatja, hogyan olvasunk be adatokat **TXT** és **CSV** fájlokból, hogyan dolgozunk az adatokkal, és hogyan írunk fájlba (pl. véletlenszámokat). A kódrészletekben **soronkénti magyarázatokat** találtak kommentekben.

1. Szükséges névterek (using-ok)

Mielőtt bármit csinálnánk, a fájl tetején meg kell adni, mely névterekből szeretnénk osztályokat használni:

```
using System; // Alap típusok: Console, Random, DateTime,
kivételek
using System.IO; // Fájlkezelés: File, StreamReader, StreamWriter,
Path
using System.Collections.Generic; // Listák (List<T>) használata
using System.Linq; // LINQ műveletek: Where, Select, OrderBy,
Average stb.
using System.Globalization; // CultureInfo – tizedespont / tizedesvessző
kezeléshez
```

 **Tipp:** Ha valamelyik osztály "ismeretlennek" tűnik a fordító szerint, általában csak egy **using** hiányzik.

2. Beolvasás TXT fájlból

2.1. Teljes fájl egyetlen szöveggént

```
// A File.ReadAllText megnyitja, beolvassa és be is zárja a fájlt egy lépésben.
// A teljes tartalmat egyetlen string-ként adja vissza, a sortörésekkel együtt.
string tartalom = File.ReadAllText("adatok.txt");

// Kiírjuk a konzolra a fájl teljes tartalmát.
Console.WriteLine(tartalom);
```

Mikor jó? Kis méretű fájloknál, ha a teljes szövegre van szükségünk egyben (pl. keresés, csere).

2.2. Sorokba olvasás (tömb)

```
// A File.ReadAllLines minden sort egy string tömb egy-egy elemébe tesz.
// sorok[0] = az 1. sor, sorok[1] = a 2. sor, stb.
string[] sorok = File.ReadAllLines("adatok.txt");

// foreach: végigmegyünk a tömb minden elemén – a "sor" változó kapja meg az
```

```
aktuális elemet.  
foreach (string sor in sorok)  
{  
    Console.WriteLine(sor);    // Kiírjuk az aktuális sort a konzolra.  
}
```

Mikor jó? Ha soronként szeretnénk feldolgozni az adatokat, és a fájl nem túl nagy (max. néhány MB).

2.3. Soronkénti olvasás StreamReader-rel (memóriabarát)

```
// A "using" blokk biztosítja, hogy a fájl automatikusan bezáruljon a blokk végén,  
// még akkor is, ha közben kivétel (hiba) keletkezik.  
using (StreamReader sr = new StreamReader("adatok.txt"))  
{  
    string? sor;    // A ? jelentése: a változó értéke lehet null is.  
  
    // A ReadLine() egyszerre csak 1 sort olvas be. Ha vége a fájlnek, null-t ad  
    // vissza.  
    // Az értékadás (sor = sr.ReadLine()) zárójelben van, így az értékét  
    // hasonlítjuk null-hoz.  
    while ((sor = sr.ReadLine()) != null)  
    {  
        Console.WriteLine(sor);    // Kiírjuk az éppen beolvasott sort.  
    }  
} // Itt a "using" blokk automatikusan meghívja sr.Dispose()-t, ami bezárja a  
fájl-t.
```

Mikor jó? Nagy fájlknál (pl. több száz MB), mert nem tölti az egészet a memóriába.

3. Beolvasás CSV fájlból

A **CSV** (Comma-Separated Values) egy szöveges formátum, ahol az értékeket elválasztó karakter (vessző , vagy pontosvessző ;) választja el. Példa: `diakok.csv`

```
Nev;Kor;Atlag  
Kovacs Anna;17;4.8  
Nagy Bela;18;3.9  
Toth Csilla;16;4.5
```

3.1. Egyszerű osztály a sorok reprezentálására

```
// Egy osztály, amely egy diákot ír le. Mindennek alapértelmezett értéket adunk,  
// hogy ne kapjunk figyelmeztetést a "null"-ról.  
class Diak  
{  
    public string Nev { get; set; } = "";    // Név – string típusú, üres a kezdő
```

```
értéke.  
    public int    Kor    { get; set; }           // Életkor – egész szám (int).  
    public double Atlag { get; set; }           // Tanulmányi átlag – lebegőpontos  
(double).  
}
```

3.2. Beolvasás és átalakítás objektumokká

```
// Létrehozunk egy üres listát, amibe a beolvasott diákokat tesszük.  
List<Diak> diakok = new List<Diak>();  
  
// Beolvassuk a teljes fájlt sorokra bontva.  
string[] sorok = File.ReadAllLines("diakok.csv");  
  
// Skip(1): átugorjuk az ELSŐ sort, mert az a fejléc (Nev;Kor;Atlag), nem adat.  
foreach (string sor in sorok.Skip(1))  
{  
    // Split(';'): a sort felvágjuk pontosvesszők mentén egy string tömbbé.  
    // pl. "Kovacs Anna;17;4.8" -> [ "Kovacs Anna", "17", "4.8" ]  
    string[] mezoK = sor.Split(';');  
  
    // Új Diak példány létrehozása, az inicializáló szintaxissal (mezőkkel  
    feltöltjük).  
    Diak d = new Diak  
    {  
        Nev    = mezoK[0],                // Az első mező egy szöveg, nem kell  
konvertálni.  
        Kor    = int.Parse(mezoK[1]),      // A második mezőt int-re alakítjuk.  
  
        // double.Parse: a harmadik mezőt double-re alakítjuk.  
        // CultureInfo.InvariantCulture: így a TIZEDESPONTOT ismeri fel  
tizedesjelnek.  
        // Magyar rendszeren különben tizedesvesszőt várna ("4,8") és hibát dobna.  
        Atlag = double.Parse(mezoK[2], CultureInfo.InvariantCulture)  
    };  
  
    diakok.Add(d); // Hozzáadjuk az új diákot a listához.  
}  
  
// Ellenőrzésképpen kiírjuk az eredményt.  
foreach (Diak d in diakok)  
{  
    Console.WriteLine($"{d.Nev} - {d.Kor} éves - átlag: {d.Atlag}");  
}
```

⚠ Figyelem: Valódi CSV fájlknál az adatok tartalmazhatnak idézőjeleket, beágyazott vesszőket, sortöréseket. Egyszerű `Split` ilyenkor nem elég — érdemes a **CsvHelper** NuGet csomagot használni.

4. Műveletek az adatokon (LINQ)

Miután a lista feltöltődött, a **LINQ** segítségével rendkívül kényelmesen szűrhetjük, rendezhetjük, összegezhethetjük az adatokat.

```
// Average: az adott tulajdonság átlagát számolja ki.
// d => d.Kor egy lambda kifejezés: minden d (Diak) elemnek vesszük a Kor mezőjét.
double atlagKor = diakok.Average(d => d.Kor);

// OrderByDescending: csökkenő sorrendbe rendezi az átlag szerint.
// First(): a legelső elemet adja vissza (vagyis a legnagyobb átlagút).
Diak legjobb = diakok.OrderByDescending(d => d.Atlag).First();

// Where: szűrés – csak azokat a diákokat tartja meg, ahol az átlag >= 4.0
// ToList(): az eredményt listává alakítjuk (a Where lustán értékel, így gyakran
kell ToList).
List<Diak> jokTanulok = diakok.Where(d => d.Atlag >= 4.0).ToList();

// Count(): megszámolja, hány elem tesz eleget a feltételnek.
int nagykoruakSzama = diakok.Count(d => d.Kor >= 18);

// Eredmények kiírása. Az F1 formátum 1 tizedesjegyre kerekít.
Console.WriteLine($"Átlag életkor: {atlagKor:F1}");
Console.WriteLine($"Legjobb tanuló: {legjobb.Nev} ({legjobb.Atlag})");
Console.WriteLine($"Jó tanulók száma (>= 4.0): {jokTanulok.Count}");
Console.WriteLine($"Nagykorúak száma: {nagykoruakSzama}");
```

5. Fájlba írás — véletlenszámok

5.1. Tömb kiírása egyszerre

```
// A Random osztály álvéletlen számokat generál.
// Egyszer példányosítjuk, NEM minden egyes szám előtt újra!
Random rnd = new Random();

// Létrehozunk egy 100 elemű string tömböt.
string[] szamok = new string[100];

// Feltöltjük 100 darab 1-100 közötti egész számmal (string-gé alakítva).
for (int i = 0; i < 100; i++)
{
    // rnd.Next(1, 101): véletlenszám 1 és 100 között (a 101 már NINCS benne).
    // .ToString(): a szám szöveggé alakítása, mert a tömb string típusú.
    szamok[i] = rnd.Next(1, 101).ToString();
}

// File.WriteAllLines: minden tömbelem egy sor lesz a fájlban.
// Ha a fájl már létezik, FELÜLÍRJA – vigyázat!
File.WriteAllLines("szamok.txt", szamok);
```

5.2. Folyamatos írás StreamWriter-rel

```
Random rnd = new Random();

// "using" – automatikus zárás. A StreamWriter létrehozza vagy felülírja a fájlt.
using (StreamWriter sw = new StreamWriter("szamok2.txt"))
{
    // 1000 véletlen szám írása soronként.
    for (int i = 0; i < 1000; i++)
    {
        int szam = rnd.Next(1, 1001); // 1 - 1000 közötti egész szám.
        sw.WriteLine(szam);           // Egy sorba írja, majd sortörést tesz a
// végére.
    }
} // Itt automatikusan bezárul a fájl.
```

5.3. Hozzáfűzés meglévő fájlhoz (append)

```
// A 2. paraméter (append: true) jelentése: NE írja felül, hanem a VÉGÉRE fűzzön.
using (StreamWriter sw = new StreamWriter("napilog.txt", append: true))
{
    // DateTime.Now: az aktuális dátum és idő.
    sw.WriteLine($"{DateTime.Now}: új bejegyzés a naplóba");
}
```

5.4. CSV írása

```
Random rnd = new Random();

using (StreamWriter sw = new StreamWriter("eredmenyek.csv"))
{
    sw.WriteLine("Sorszam;Ertek"); // Fejléc – ez lesz az első sor.

    // 50 sor adat kiírása.
    for (int i = 1; i <= 50; i++)
    {
        // rnd.NextDouble(): véletlen double 0.0 és 1.0 között.
        // *100 -> 0.0 és 100.0 közötti tartomány.
        double erteK = rnd.NextDouble() * 100;

        // "F2": 2 tizedesjegyre kerekítve formázzuk.
        // InvariantCulture: tizedesPONTtal írjuk ki (nemzetközi formátum).
        string erteKszoveg = erteK.ToString("F2", CultureInfo.InvariantCulture);

        // Kiírjuk a sort: sorszám ; érték
        sw.WriteLine($"{i};{erteKszoveg}");
    }
}
```

```
}  
}
```

6. Hibakezelés (try-catch)

A fájlműveletek sok minden miatt hibára futhatnak: nincs fájl, foglalt, nincs jogunk hozzá stb. Éles kódban érdemes **try-catch** blokkba tenni őket.

```
try  
{  
    // A "veszélyes" műveletek ide kerülnek.  
    string[] sorok = File.ReadAllLines("adatok.txt");  
    Console.WriteLine($"Sikeresen beolvasva {sorok.Length} sor.");  
}  
catch (FileNotFoundException)  
{  
    // Akkor fut le, ha a fájl nem létezik.  
    Console.WriteLine("Hiba: a fájl nem található.");  
}  
catch (UnauthorizedAccessException)  
{  
    // Akkor fut le, ha nincs jogosultságunk a fájlhoz.  
    Console.WriteLine("Hiba: nincs jogosultság a fájlhoz.");  
}  
catch (IOException ex)  
{  
    // Általános I/O hiba (pl. lemez tele, fájl foglalt).  
    // Az ex.Message a hiba szövegét tartalmazza.  
    Console.WriteLine($"I/O hiba: {ex.Message}");  
}
```

Előzetes ellenőrzés

```
// Sokszor egyszerűbb előre megkérdezni: létezik-e a fájl?  
if (File.Exists("adatok.txt"))  
{  
    string[] sorok = File.ReadAllLines("adatok.txt");  
    // feldolgozás...  
}  
else  
{  
    Console.WriteLine("A fájl nem létezik!");  
}
```

7. Hasznos tippek

7.1. Elérési utak

```
// A "adatok.txt" RELATÍV út – a program futási könyvtárához (bin/Debug/netX.X/)
képest értelmezett.
// Path.Combine: biztonságosan összerakja az útvonalat (Windows / Linux
átjárhatóan).
string ut = Path.Combine(Environment.CurrentDirectory, "adatok.txt");

// Abszolút útvonal megadása példa (Windows):
string abszolutUt = @"C:\Adatok\diakok.csv";
// A @ jel előtag jelentése: "verbatim string" – a backslash-t NEM kell duplázni.
```

7.2. Karakterkódolás

```
// Magyar ékezetes karakterekhez gyakran UTF-8 kódolást használunk.
// Olvasáskor:
string[] sorok = File.ReadAllLines("adatok.txt", System.Text.Encoding.UTF8);

// Íráskor:
File.WriteAllText("kimenet.txt", "Árvíztűrő tükörfúrógép",
System.Text.Encoding.UTF8);
```

7.3. Mikor melyik módszert?

Helyzet	Ajánlott módszer
Kicsi fájl, egyszerű feldolgozás	<code>File.ReadAllLines</code> / <code>File.ReadAllText</code>
Nagy fájl, soronkénti feldolgozás	<code>StreamReader</code> + <code>while</code>
Egyszerű kiírás listából	<code>File.WriteAllLines</code>
Sok adat, fokozatosan keletkezik	<code>StreamWriter</code> + <code>using</code>
Naplózás (új bejegyzés a végére)	<code>StreamWriter</code> + <code>append: true</code>

8. Összefoglaló mini-projekt

Egy program, amely **véletlenszámokat generál fájlba, majd visszaolvassa és statisztikát készít** róluk:

```
using System;
using System.IO;
using System.Linq;

class Program
{
    static void Main()
    {
```

```
string fajl = "szamok.txt";
Random rnd = new Random();

// 1) ÍRÁS: 100 véletlen szám fájlba.
using (StreamWriter sw = new StreamWriter(fajl))
{
    for (int i = 0; i < 100; i++)
    {
        sw.WriteLine(rnd.Next(1, 101)); // 1 - 100 közötti szám.
    }
}

// 2) OLVASÁS: a fájl tartalmának visszaolvasása.
string[] sorok = File.ReadAllLines(fajl);

// 3) ÁTALAKÍTÁS: a string sorokat int-é alakítjuk LINQ-val.
//     Select: minden elemen elvégzünk egy átalakítást.
//     int.Parse: szöveg -> egész szám.
int[] szamok = sorok.Select(s => int.Parse(s)).ToArray();

// 4) STATISZTIKA: alap értékek.
Console.WriteLine($"Darabszám: {szamok.Length}");
Console.WriteLine($"Minimum:    {szamok.Min()}");
Console.WriteLine($"Maximum:    {szamok.Max()}");
Console.WriteLine($"Átlag:      {szamok.Average():F2}");
Console.WriteLine($"Összeg:     {szamok.Sum()}");

// 5) Hány szám esik 50 fölé?
int nagyok = szamok.Count(x => x > 50);
Console.WriteLine($"50 fölötti: {nagyok} db");
}
```

✓ Mit gyakoroljatok?

1. Olvassatok be egy szöveges fájlt, és számoljátok meg, hány szó van benne!
2. Készítsetek CSV fájlt termékadatokkal (név, ár, raktárkészlet), és listázzátok ki azokat, amelyekből kevesebb mint 10 db van!
3. Generáljatok 1000 véletlen lottószámot (1-90 között) fájlba, majd számoljátok meg, melyik szám hányszor fordult elő!
4. Készítsetek egyszerű naplózót, ami egy `napilog.txt` fájlba minden indításkor új bejegyzést ír a dátummal!